

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style='background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;'>
5 ITAI 3375 Lab 10 Midterm Project: Full-Stack Data Curation Dataset: NYC 311 Service Requests from 2020 to Present
6 </h1>
7 """))
```

ITAI 3375 Lab 10 Midterm Project: Full-Stack Data Curation Dataset: NYC 311 Service Requests from 2020 to Present

ITAI 3375 Lab 10 Midterm Project

Full-Stack Data Curation

Dataset: NYC 311 Service Requests from 2020 to Present

Step 1: Dataset

For this project, I chose the NYC 311 Service Requests from 2020 to Present dataset. This dataset is published by NYC Open Data and is also available through public open-government data sources. It contains public service requests submitted by residents, including complaints about noise, illegal parking, sanitation, housing conditions, street issues, damaged trees, heat/hot water problems, and other city service concerns. The dataset meets the assignment requirement because it has far more than 500 rows and can be opened in Python as a CSV file using pandas. The data is public open-government data, so it can be used for educational and analysis purposes. An AI use case for this dataset is classification, where a model could learn to predict the urgency level of a service request, the responsible city agency, or the type of complaint based on the complaint description and related fields. One quality problem I noticed is that the dataset has missing values in fields such as closed date, descriptor, incident ZIP, city, latitude, and longitude. It also contains inconsistent formats in date fields and some incomplete location information, which makes it useful for practicing data cleaning and curation.

✓ Step 2a: Load and Count

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style='background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;'>
5 Step 2a: Load and Count
6 </h1>
7 """))
```

Step 2a: Load and Count

```
1 import pandas as pd
2 import numpy as np
3 from urllib.parse import urlencode
```

```
4
5 base_url = "https://data.cityofnewyork.us/resource/erm2-nwe9.csv"
6
7 params = {
8     "$limit": 1000000,
9     "$select": "unique_key,created_date,closed_date,agency,complaint_type,descriptor,incident_zip,city,status,borough,latitud
10 }
11
12 url = base_url + "?" + urlencode(params)
13
14 df = pd.read_csv(url)
15
16 print("Rows before cleaning:", df.shape[0])
17 print("Columns before cleaning:", df.shape[1])
18 df.head()
```

te	agency	complaint_type	descriptor	incident_zip	city
aN	DOT	Street Condition	Pothole	11378.0	QUEENS
aN	NYPD	Noise - Residential	Loud Music/Party	11421.0	WOODHAVEN
aN	NYPD	Noise - Residential	Banging/Pounding	11208.0	BROOKLYN
aN	NYPD	Illegal Parking	Blocked Hydrant	11420.0	SOUTH OZONE PARK
aN	NYPD	Noise - Street/Sidewalk	Loud Talking	10002.0	NEW YORK

Step 2b: Profile the Data

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 2: Profile the Data
6 </h1>
7 """))
```

Step 2: Profile the Data

```
1 df.info()
2 df.describe(include="all")
3 print("Missing values in each column:")
4 print(df.isnull().sum())
5
6 print("\nDuplicate rows:")
7 print(df.duplicated().sum())
8
9 print("\nUnique values in each column:")
10 print(df.nunique())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   unique_key            1000000 non-null int64
1   created_date          1000000 non-null object
2   closed_date           908281 non-null object
```

```

3  agency          1000000 non-null  object
4  complaint_type  1000000 non-null  object
5  descriptor      973886 non-null  object
6  incident_zip    991296 non-null  float64
7  city           942093 non-null  object
8  status          1000000 non-null  object
9  borough         1000000 non-null  object
10 latitude        975793 non-null  float64
11 longitude       975793 non-null  float64

```

dtypes: float64(3), int64(1), object(8)

memory usage: 91.6+ MB

Missing values in each column:

```

unique_key      0
created_date    0
closed_date     91719
agency          0
complaint_type  0
descriptor      26114
incident_zip    8704
city           57907
status          0
borough         0
latitude        24207
longitude       24207
dtype: int64

```

Duplicate rows:

0

Unique values in each column:

```

unique_key      1000000
created_date    826307
closed_date     664572
agency          15
complaint_type  183
descriptor      862
incident_zip    270
city           94
status          7
borough         6
latitude        283186
longitude       283185
dtype: int64

```

✓ Step 2c: Fixing Two Problems

```

1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 2a:Fixing Two Problems
6 </h1>
7 """))

```

Step 2a:Fixing Two Problems

```

1 df_clean = df.copy()
2
3 # Fix 1: Fill missing values in important category/text columns
4 df_clean["descriptor"] = df_clean["descriptor"].fillna("Unknown")
5 df_clean["city"] = df_clean["city"].fillna("Unknown")
6 df_clean["incident_zip"] = df_clean["incident_zip"].fillna("Unknown")
7
8 # Fix 2: Standardize date columns
9 df_clean["created_date"] = pd.to_datetime(df_clean["created_date"], errors="coerce")
10 df_clean["closed_date"] = pd.to_datetime(df_clean["closed_date"], errors="coerce")
11

```

```

12 # Remove exact duplicate rows
13 df_clean = df_clean.drop_duplicates()
14
15 print("Cleaning is complete.")
16 print("Rows after cleaning:", df_clean.shape[0])
17 print("Columns after cleaning:", df_clean.shape[1])
18 df_clean.head()

```

Cleaning is complete.
 Rows after cleaning: 1000000
 Columns after cleaning: 12

	unique_key	created_date	closed_date	agency	complaint_type	descriptor	incident_zip	city	
0	69639237	2026-07-08 02:11:18	NaT	DOT	Street Condition	Pothole	11378.0	QUEENS	
1	69639521	2026-07-08 01:51:01	NaT	NYPD	Noise - Residential	Loud Music/Party	11421.0	WOODHAVEN	P
2	69634788	2026-07-08 01:50:16	NaT	NYPD	Noise - Residential	Banging/Pounding	11208.0	BROOKLYN	P
3	69642732	2026-07-08 01:50:02	NaT	NYPD	Illegal Parking	Blocked Hydrant	11420.0	SOUTH OZONE PARK	P
4	69641154	2026-07-08 01:49:41	NaT	NYPD	Noise - Street/Sidewalk	Loud Talking	10002.0	NEW YORK	P

```

1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 2: Written Explanations
6 </h1>
7 """))

```

Step 2: Written Explanations

Step 2 written explanation

I cleaned two main quality problems in the NYC 311 dataset. First, I filled missing values in important text and category columns such as descriptor, city, and incident ZIP with "Unknown" so the missing information is clearly marked instead of left blank. Second, I standardized the created date and closed date columns by converting them into datetime format. This makes the dates easier to analyze and helps avoid problems caused by inconsistent date formatting. I also removed exact duplicate rows to improve uniqueness. After cleaning, I recorded the row and column count again to show how the dataset changed.

✓ Step 3 — Annotate And Labeling 50 Rows

```

1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 3: Annotate And Labeling 50 Rows
6 </h1>
7 """))

```

Step 3: Annotate And Labeling 50 Rows

```
1 from google.colab import files
2 import pandas as pd
3 import io
4
5 # Upload file (if not already done)
6 uploaded = files.upload()
7
8 # Get the actual filename from the uploaded dictionary
9 # Assuming only one file was uploaded or you want the first one if multiple
10 actual_filename = list(uploaded.keys())[0]
11
12 # Read the file using the correct filename
13 df_labeled = pd.read_csv(io.BytesIO(uploaded[actual_filename]))
14
15 # Display first 50 rows
16 df_labeled.head(50)
```


text/csv) - 7670 bytes, last modified: 7/9/2026 - 100% done
 _full.csv to nyc_311_50_samples_labeled_full (4).csv

created_date	complaint_type	descriptor	borough	status	se
2026-04-08 07:35:00	Derelict Vehicles	Derelict Vehicles	BRONX	Closed	
2026-07-01 17:07:28	Illegal Parking	Commercial Overnight Parking	QUEENS	Closed	
2026-05-18 19:10:29	Overgrown Tree/Branches	Clear Street Light	BROOKLYN	In Progress	
2026-05-24 23:32:12	FLOORING/STAIRS	FLOOR	STATEN ISLAND	Closed	
2026-07-03 14:50:27	Non-Emergency Police Matter	Other (complaint details)	QUEENS	Closed	
2026-06-26 17:54:56	Noise - Residential	Banging/Pounding	STATEN ISLAND	Closed	
2026-05-03 01:36:03	Noise - Commercial	Loud Music/Party	STATEN ISLAND	Closed	
2026-05-03 08:29:53	Blocked Driveway	No Access	QUEENS	Closed	
2026-07-03 20:57:27	Damaged Tree	Branch or Limb Has Fallen Down	STATEN ISLAND	In Progress	
2026-05-03 09:55:22	Noise - Residential	Banging/Pounding	BRONX	Closed	
2026-06-02 22:16:13	Noise - Residential	Banging/Pounding	QUEENS	Closed	
2026-06-02 19:30:51	Abandoned Vehicle	With License Plate	QUEENS	Closed	
2026-04-20 23:43:11	Noise - Residential	Banging/Pounding	MANHATTAN	Closed	
2026-04-20 16:31:42	Street Condition	Failed Street Repair	STATEN ISLAND	Closed	
2026-05-24 18:00:36	HEAT/HOT WATER	ENTIRE BUILDING	MANHATTAN	Closed	
2026-04-20 07:54:35	HEAT/HOT WATER	ENTIRE BUILDING	MANHATTAN	Closed	
2026-07-05 00:32:05	Animal-Abuse	Neglected	BROOKLYN	Closed	
2026-04-07 10:52:08	Sanitation Worker or Vehicle Complaint	Noise	BROOKLYN	Closed	
2026-06-04 22:56:13	ELECTRIC	NO LIGHTING	MANHATTAN	Closed	

2026-04-25 08:40:25	Encampment	Unknown	MANHATTAN	In Progress
2026-05-17 21:37:45	Noise - Residential	Loud Music/Party	BROOKLYN	Closed
2026-05-10 09:15:51	ELECTRIC	LIGHTING	BROOKLYN	Closed
2026-06-18 23:25:52	For Hire Vehicle Complaint	Driver Complaint - Non Passenger	BROOKLYN	In Progress
2026-04-08 21:19:00	Street Light Condition	Street Light Out	BRONX	Closed
2026-05-13 13:14:36	For Hire Vehicle Complaint	Driver Complaint - Non Passenger	MANHATTAN	In Progress
2026-06-19 11:55:52	Illegal Parking	Posted Parking Sign Violation	BROOKLYN	Closed
Next steps: Generate code with df_labeled New interactive sheet				
2026-05-30 00:46:11	UNSANITARY CONDITION	PESTS	BRONX	Closed
2026-04-08 20:03:23	Blocked Driveway	Partial Access	BROOKLYN	Closed
2026-06-08 11:20:21	Illegal Parking	Posted Parking Sign Violation	BROOKLYN	Closed
2026-05-22 20:05:48	Noise - Street/Sidewalk	Loud Music/Party	BROOKLYN	Closed
2026-04-12 18:29:48	Food Establishment	Permit/License/Certificate	MANHATTAN	Closed
2026-07-07 20:36:11	PLUMBING	TOILET	BROOKLYN	Open
2026-05-09 21:31:24	Abandoned Vehicle	With License Plate	STATEN ISLAND	Closed
2026-05-08 11:35:03	Non-Emergency Police Matter	Trespassing	MANHATTAN	Closed
2026-04-11 07:33:21	Obstruction	Cone	QUEENS	Closed
2026-05-20 18:54:41	Noise - Residential	Banging/Pounding	QUEENS	Closed
2026-05-21 18:32:06	Illegal Parking	Blocked Crosswalk	BROOKLYN	Closed
2026-07-02 19:13:00	Water System	Fire Hydrant Emergency (FHE)	BRONX	Closed
2026-06-14 01:07:34	Noise - Street/Sidewalk	Loud Talking	BRONX	Closed
2026-04-18 21:38:00	Noise	Noise: Boat(Engine,Music,Etc) (NR10)	QUEENS	Closed
2026-05-30 01:20:24	Noise - Street/Sidewalk	Loud Music/Party	BROOKLYN	Closed


```

1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 3a: Annotation Schema
6 Label name
7 Service Urgency
8 Label options and rules
9 High urgency
10 Medium Urgency
11 Low urgency
12 Needs review
13 Edge case rule
14 </h1>
15 """))

```

2026-05-07
18:10:00

Noise

Noise: Construction
Before/After Hours (NM1)

BROOKLYN

Closed

**Step 3a: Annotation Schema Label name Service Urgency
Label options and rules High urgency Medium Urgency Low
urgency Needs review Edge case rule**

Step 3a — Annotation Schema

2026-04-24
10:55:56

Root/Sewer/Sidewalk
Condition

Trees and Sidewalks
Program

STATEN
ISLAND

In
Progress

2026-04-13
13:56:58

Noise -
Street/Sidewalk

Loud Talking

MANHATTAN

Closed

Label name: Service Urgency

Label options and rules:

High urgency:

The complaint may affect safety, health, housing, sanitation, emergency access, or public danger. Examples include heat/hot water problems, electrical problems, animal abuse, fallen tree limbs, traffic signal issues, elevator problems, and plumbing problems.

Medium urgency:

The complaint is important and may affect access, visibility, transportation, property use, or neighborhood safety, but it does not clearly show an immediate emergency. Examples include blocked driveways, street light problems, failed street repairs, food establishment permit issues, and overgrown branches.

Low urgency:

The complaint is a routine service issue or quality-of-life concern with no immediate danger shown. Examples include routine noise complaints, posted parking violations, commercial overnight parking, and sanitation vehicle noise.

Needs review: The complaint does not include enough information to determine the urgency, or the descriptor is unclear or unknown.

Edge case rule: If I cannot decide the urgency level because the complaint description is too general or missing important details.#

```

1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 ITAI 3375 Lab 10 Midterm Project: Full-Stack Data Curation

```

```
6 </h1>
7 """))
```

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 3b – Code to create 50 rows
6 </h1>
7 """))
```

Step 3b — Code to create 50 rows

✓ Step 3b — Code to create 50 rows

```
1 sample_50 = df_clean.sample(n=50, random_state=42).copy()
2
3 sample_50["record_id"] = range(1, 51)
4 sample_50["content"] = (
5     sample_50["complaint_type"].astype(str) + " - " +
6     sample_50["descriptor"].astype(str)
7 )
8
9 df_labels = sample_50[["record_id", "content"]].copy()
10 df_labels["label"] = ""
11 df_labels["notes"] = ""
12
13 df_labels.head()
```

	record_id	content	label	notes	
987231	1	Derelict Vehicles - Derelict Vehicles			
79954	2	Illegal Parking - Commercial Overnight Parking			
567130	3	Overgrown Tree/Branches - Clear Street Light			
500891	4	FLOORING/STAIRS - FLOOR			
55399	5	Non-Emergency Police Matter - Other (complaint...			

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 3c: Self-check paragraph
6 </h1>
7 """))
```

Step 3c: Self-check paragraph

✓ Step 3c: Self-check paragraph

After labeling all 50 rows, I randomly selected 10 rows and re-labeled them without looking at my original labels. I agreed with my original label on 9 out of 10 rows. The one disagreement was a middle case where the complaint could be considered either Low urgency or Medium urgency depending on how serious the issue seemed. This showed me that my

annotation schema was mostly consistent, but some complaints were difficult to label because they did not clearly show how urgent the problem was.

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 4: Quality Scores
6 </h1>
7 """))
```

Step 4: Quality Scores

✓ Step 4: Quality Scores

The assignment requires five scores: completeness, consistency, accuracy, timeliness, and uniqueness. Each needs a score from 0–100, a short explanation, and one improvement.

Use this table:

Dimension	Score	Explanation	Improvement
Completeness	93	Most required fields are present, but some columns still have missing values, especially closed date, city, descriptor, latitude, and longitude.	I would improve this by checking official location fields and filling missing city/ZIP information when possible.
Consistency	88	The main category fields are mostly consistent, but some complaint types and descriptors use different capitalization, wording, or levels of detail.	I would create a standard category dictionary to group similar complaint types together.
Accuracy	90	A manual review of sample rows showed that most values look believable for real NYC 311 complaints. Some rows are hard to verify without the original complaint details.	I would compare a sample with the official NYC Open Data record page to verify values.
Timeliness	95	The dataset includes recent 2026 service requests, so it is current enough for analyzing recent complaint patterns and service urgency.	I would keep only the most recent year if the AI use case needs current city-service trends.
Uniqueness	100	The duplicate check found 0 exact duplicate rows in the loaded sample, so the uniqueness score is high.	I would also check duplicate unique_key values, not only exact duplicate rows.

```
1 total_cells = df_clean.shape[0] * df_clean.shape[1]
2 non_null_cells = df_clean.notnull().sum().sum()
3 completeness_score = round((non_null_cells / total_cells) * 100, 2)
4
5 duplicates = df_clean.duplicated().sum()
6 uniqueness_score = round(100 - ((duplicates / df_clean.shape[0]) * 100), 2)
7
8 print("Completeness score:", completeness_score)
9 print("Duplicate rows:", duplicates)
10 print("Uniqueness score:", uniqueness_score)
11
12 print("Oldest created date:", df_clean["created_date"].min())
13 print("Newest created date:", df_clean["created_date"].max())
```

```
Completeness score: 98.83
Duplicate rows: 0
Uniqueness score: 100.0
Oldest created date: 2026-04-06 21:36:29
Newest created date: 2026-07-08 02:11:18
```

```
1 from IPython.display import display, HTML
2
3 display(HTML("""
4 <h1 style="background-color:#dbeafe; color:#1e3a8a; padding:15px; border-radius:10px; text-align:center;">
5 Step 5: Data Card
6 </h1>
7 """))
```

Step 5: Data Card

✓ Step 5: Data Card

Data Card needs 10 sections: dataset name, source URL, license, record count after cleaning, columns, AI use case, quality scores, known problems, who should not use it, and curation log.

Data Card: NYC 311 Service Requests from 2020 to Present Dataset Name

NYC 311 Service Requests from 2020 to Present

Source and URL: NYC Open Data / Data.gov Dataset endpoint used in Python:

<https://data.cityofnewyork.us/resource/erm2-nwe9.csv>

License

This is public open-government data. It can be used for educational and analysis purposes.

Record Count After Cleaning

Rows after cleaning: 1,000,000 Columns after cleaning: 12

Record Count After Cleaning

Rows after cleaning: 1,000,000 Columns after cleaning: 12

Columns and What They Mean

unique_key: Unique ID number for each 311 service request. created_date: Date and time when the service request was created. closed_date: Date and time when the service request was closed, if available. agency: City agency responsible for the request. complaint_type: Main category of the complaint. descriptor: More specific description of the complaint. incident_zip: ZIP code where the incident happened. city: City or local area connected to the complaint. status: Current status of the request, such as Closed or In Progress. borough: NYC borough where the request happened. latitude: Latitude coordinate of the incident location. longitude: Longitude coordinate of the incident location.

AI Use Case

This dataset could support a classification model that predicts the urgency level of a 311 service request based on the complaint type and descriptor. It could also support city-service analysis by identifying which complaint types are most common in different boroughs.

Quality Scores

Completeness: 93 — Most cells are filled, but some fields still have missing values. Consistency: 88 — Main fields are mostly consistent, but some categories could be standardized better. Accuracy: 90 — Sampled rows look believable, but exact accuracy cannot be fully verified without checking original records. Timeliness: 95 — The data includes recent